

Módulo 6: Infraestructura y Deploy

T17: Dockerización Segura

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Usuario no-root, imagen mínima y secretos en tiempo de ejecución*

Objetivos

- Explicar los riesgos de ejecutar contenedores como `root`
- Crear una imagen Docker segura: usuario no-root, imagen `node:alpine`, multi-stage build
- Gestionar secretos en Docker correctamente (ENV vs. build-args vs. secrets)
- Escanear la imagen con `docker scout` o Trivy para detectar CVEs

El riesgo de ejecutar como root

```
# ✗ Dockerfile inseguro (por defecto, todo es root)
FROM node:20
COPY . /app
RUN npm install
CMD ["node", "dist/index.js"]
```

Si hay un RCE (Command Injection, SSRF...):

- El atacante es `root` dentro del contenedor
- Puede montar volumes, acceder a secretos, escapar del contenedor
- `docker.sock` montado = acceso total al host

Dockerfile seguro — buenas prácticas

```
#  Multi-stage build + usuario no-root
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM node:20-alpine AS runtime
WORKDIR /app

# Crear usuario sin privilegios
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copiar solo lo necesario
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json .

# Ejecutar como usuario no-root
USER appuser
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Secretos en Docker — cómo NO hacerlo vs. cómo sí

```
# ❌ NUNCA meter secretos en la imagen
ENV JWT_SECRET=mi-secreto-super-seguro
# Visible con: docker inspect / docker history
```

```
# ❌ NUNCA usar build-args para secretos
ARG DB_PASSWORD
# Quedan en las capas de la imagen
```

```
# ✅ Secretos en runtime (docker-compose)
services:
  app:
    env_file: .env           # Opción 1: archivo .env
    secrets:                 # Opción 2: Docker secrets
      - jwt_secret

secrets:
  jwt_secret:
    file: ./secrets/jwt.txt
```

Imagen mínima = menos superficie de ataque

Imagen base	Tamaño	Paquetes	Riesgo
<code>node:20</code>	~1 GB	Debian completo	Alto
<code>node:20-slim</code>	~200 MB	Debian mínimo	Medio
<code>node:20-alpine</code>	~130 MB	Alpine (musl)	Bajo
<code>gcr.io/distroless/nodejs20</code>	~120 MB	Solo runtime	Mínimo

✓ **Menos paquetes = menos CVEs potenciales.** Usar `alpine` o `distroless` siempre que sea posible.

Escaneo de imágenes

Docker Scout (integrado en Docker Desktop)

```
docker scout cves local://vulnapp:latest
```

Trivy (open source)

```
trivy image vulnapp:latest
```

Snyk Container

```
snyk container test vulnapp:latest
```

Herramienta	Tipo	Integración CI
Docker Scout	Integrado	GitHub Actions
Trivy	Open source	Cualquier CI
Snyk	SaaS	GitHub/GitLab/Azure

Checklist de Docker seguro

Aspecto	Inseguro	Seguro
Usuario	root (default)	USER appuser
Imagen	node:20 (1GB)	node:20-alpine (130MB)
Build	Single stage	Multi-stage
Secretos	ENV / ARG	Runtime env_file / Docker secrets
.dockerignore	No existe	Excluye .env , node_modules , .git
Escaneo	Nunca	CI pipeline con Trivy/Scout

Capabilities y read-only filesystem

```
# docker-compose.yml - hardening avanzado
services:
  app:
    image: vulnapp:latest
    read_only: true           # ← Filesystem de solo lectura
    tmpfs:
      - /tmp                 # Solo /tmp es escribible
    cap_drop:
      - ALL                  # Quitar TODAS las capabilities
    cap_add:
      - NET_BIND_SERVICE    # Solo lo mínimo necesario
    security_opt:
      - no-new-privileges:true # No puede escalar privilegios
```

Impacto si hay RCE:

- ✗ No puede escribir en disco (binarios, backdoors)
- ✗ No puede montar filesystems ni cambiar permisos
- ✗ No puede hacer `setuid` / `setgid`
- ✓ Solo puede usar la red (pero limitada por network policies)

✓ `read_only: true` + `cap_drop: ALL` reduce drásticamente el impacto de un RCE.

.dockerignore — lo que NUNCA debe entrar en la imagen

```
# .dockerignore
.env
.env.*
.git
.gitignore
node_modules
*.md
tests/
coverage/
docker-compose*.yaml
secrets/
```

💀 Sin `.dockerignore`, un `COPY . .` mete `.env`, `.git`, y `node_modules` enteros en la imagen — exponiendo secretos y aumentando superficie.

Lab T17: Endureciendo el Dockerfile de VulnApp

Objetivo: crear un Dockerfile seguro para VulnApp

Setup: `cd VulnApp`

1. Revisa el `Dockerfile` existente — ¿usa root? ¿imagen base grande?
2. Añade un usuario no-root: `RUN addgroup -S app && adduser -S app -G app`
3. Implementa multi-stage build para reducir el tamaño de la imagen final
4. Escanea la imagen: `docker scout cves local://vulnapp:latest`